Assignment

Machine Learning Concepts
Artificial Intelligence

Lecturer: Luciano Gerber

Muhammad Zahid
25925501

January 2026

1 Data/Domain Understanding and Exploration

1.1 Meaning and Type of Features; Analysis of Distributions

The provided dataset is focused on **Car Sales Adverts** provided by **AutoTraders**. The information is focused on vehicle information such as mileage, brand, type, color, and price. The main task is to use this information to predict the price of the cars. As such the dataset contains mixture of numerical, categorical, and binary features, each with distinct effects on the prediction capabilities on price.

Feature Types and Semantics: The **target variable** Price, is a continuous numerical feature. Initial inspection of this via a hist-plot Figure 1 reveals price to be highly right skewed, which could be the presence of outliers. But this is a reflection of Real-World markets where luxury vehicles are rare and pricey, dealing with them can be tricky. The dataset itself only contains two meaningful numeric features:

- **mileage**: Indication of usage of vehicle. Expected to have negative correlation to price.
- **Year of Registration**: Indication of how new the vehicle is. Expected to have a positive correlation to price.

Categorical Features on the other hand are of varying types based on cardinality:

- **Low-cardinality categorical features**: Contains general characteristic features like **fuel type** and **body type**.
- **Binary categorical features**: Contains vehicle condition and classifications type features; **vehicle condition** and **cross over car and van**.
- **High-cardinality categorical features**: Contains model specific or brand specific features like **standard make**, **standard model**, and **standard colour**.

Distributional Analysis: Based on the skewness of the price could cause regression models to weighting heavily on extremely priced vehicles. For this purpose we refer to log transformation method (**log1p**) for normalization as shown in Figure 2. On the other hand Figure 3 shows a long right tail, with most of the vehicles being concentrated below 150,000 to 200,000 range of mileage.

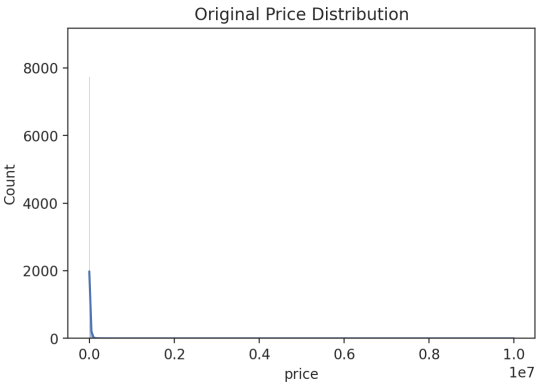


Figure 1: Price Hist-Plot

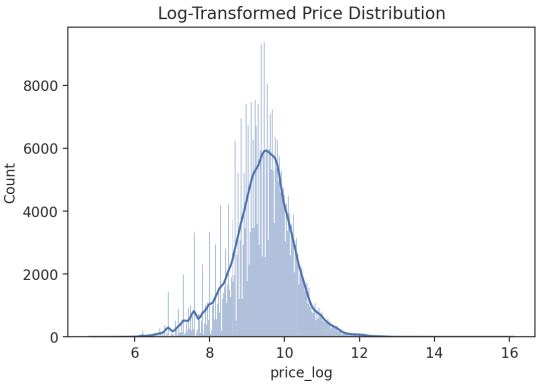


Figure 2: Log-Price Hist-plot

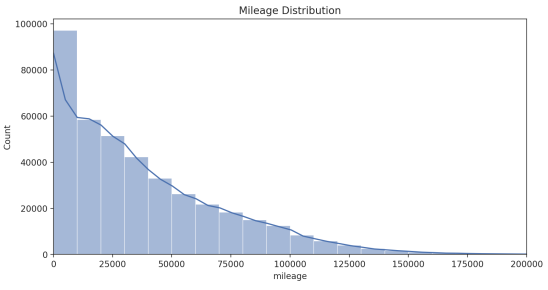
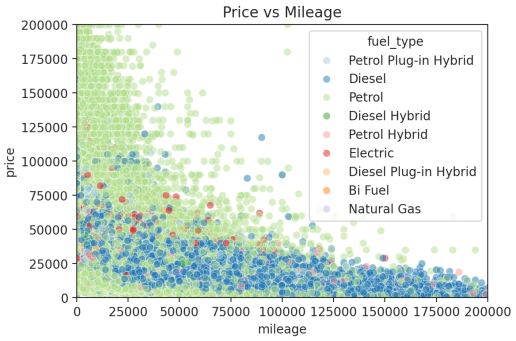


Figure 3: Mileage Histogram

1.2 Analysis of Predictive Power of Features

To access the predictive power of the features, exploratory analysis like heat-maps where conducted for this purpose using:

```
plt.figure(figsize=(10, 8))
sns.heatmap(AT_demo[['mileage', 'year_of_registration', 'price_log']].co
plt.title('Correlation Heatmap of Numerical Features')
plt.show()
```



fmt='.2f'

Figure 4: Caption

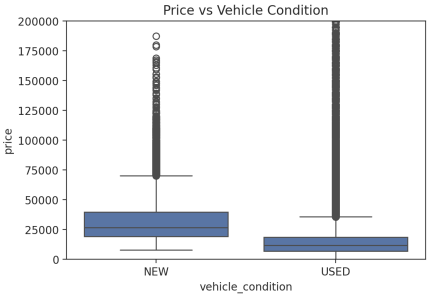


Figure 5: Vehicle condition

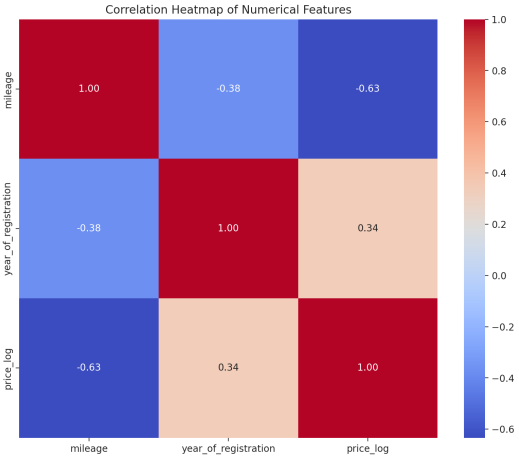


Figure 6: Heat-map

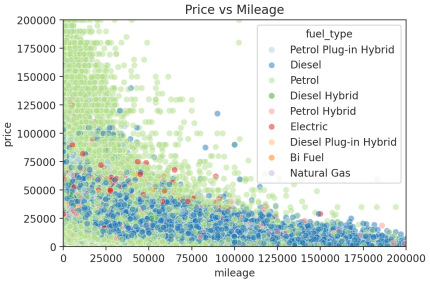


Figure 7: Fuel Scatter Plot

According to the heat-map a shown in Figure 6 the relationship between price and mileage is as we predicted negatively correlated (-0.63), while the relationship between price and year of registration is positive with a 0.34 value according to the heat-map.
Categorical Features also exhibit strong predictive behavior for example, according to the Figure 5 vehicles that are new and in better condition give a relatively higher median value compared to the used vehicles, with far less variance.
Although the effect is not as major, another feature is the fuel type and the effect it has is shown in the scatter plot in Figure 7. Majority of the petrol and diesel hybrid cover the upper and lower price ranges with low mileage, while the Diesel and Petrol plug in hybrid take the lower price ranges between 0 to 75,000 with higher mileage.
For standard model and standard make median price comparison revealed the top 20 most popular models and brands. Other than that not much additional information is available than what we already know.
Overall, the exploratory analysis suggests a combination of categorical (vehicle condition, model, brand)and numerical features (mileage and year of registration) to be strongly influential in prediction of prices. This suggest the need for appropriate feature encoding in different model stages and flexible regression models for price prediction.

1.3 Data Processing for Exploration and Visualization

For the purpose of data processing we must first visualize the inconsistencies in the data set. The Table. 1 gives an idea of how many missing values in the dataset we are dealing with, half of the data set columns have missing values. Considering the rows and columns being (402005, 12) these missing values are not by any means a major headache and simply dropping the will do the trick, but we must deal with them professionally and in doing so can help the models prediction capabilities. A simple chart of missing values can be generated with a simple code:

```
AT.isnull().sum().plot(kind='bar')
plt.title('Missing Values Bar Chart')
plt.ylabel('Missing Values Count')
plt.show()
```

The resulting Bar Chart in Figure 8 tells us only two columns have major missing values reg code and year of registration, meaning only these two will effect the prediction capabilities of our model, the rest can be dealt with simple imputing median or mean. Upon closer inspection of reg code column we find that according to the UK license plate policies these number plate imputed in the dataset are only the year and potentially season (start of year or end of year) they were registered. Considering that this information is already available in year of registration this column is not very useful. Other than that getting information such as season, special number plates, and expensive number plates etc can be a major boon for us. But the inconsistencies in this feature dealing with the missing values are difficult and time consuming. Hence, we will drop this column and use year of registration in place of this.

On the other hand the feature year of registration also has missing values as shown in Figure 8, as well as inconsistencies visible in Figure 9. The inconsistencies were visualized through a scatter plot helping us identifying outliers in the dataset:

```
sns.scatterplot(x='year_of_registration', y='price', data=AT, alpha=0.3)
plt.title('Price vs year_of_registration')
plt.show()
```

Observations tell us some values are in year 1000 which is impossible by checking the first license plate registration we can identify the first year of registration 1903, we can use this for dealing with outliers. For the purpose of Machine Learning these steps must be taken for data pre-processing before feeding these feature to the model.

columns	Before	After
public_reference	0	0
mileage	127	0
reg_code	31857	0
standard_colour	5378	0
standard_make	0	0
standard_model	0	0
vehicle_condition	0	0
year_of_registration	33311	0
price	0	0
body_type	837	0
crossover_car_and_van	0	0
fuel_type	601	0

Table 1: Missing Values Columns

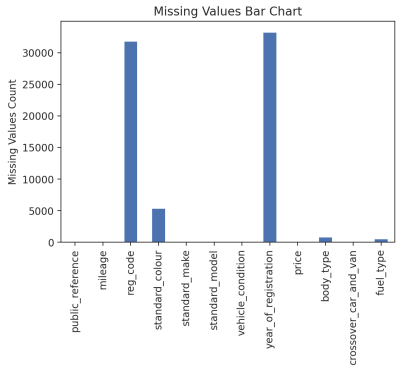


Figure 8: Missing Value Chart

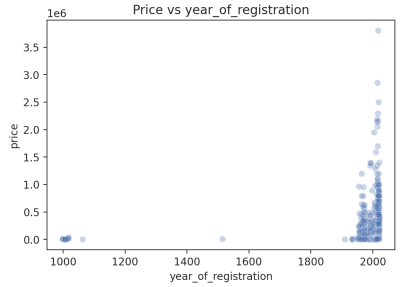


Figure 9: Year Of Registration

2 Data Processing for Machine Learning

2.1 Missing Values, Outliers, and Noise

Three steps imputation strategy has been implemented for dealing with year of registration column. First by grouping using **standard model** feature a column that has zero missing values and computing missing values in year of registration through the median value:

```
AT_copy['year_of_registration'] = AT_copy['year_of_registration'].fillna(
    AT_copy.groupby('standard_model')['year_of_registration'].transform('median')
)
```

The First step imputes most of the missing values but not all, so a Second step using the same method but a different feature namely standard make is used:

```
AT_copy['year_of_registration'] = AT_copy['year_of_registration'].fillna(
    AT_copy.groupby('standard_make')['year_of_registration'].transform('median')
)
```

For the last step, if still any remaining missing values are detected a simple median of the year of registration column fills these missing values:

```
AT_copy['year_of_registration'] = AT_copy['year_of_registration'].fillna(
    AT['year_of_registration'].median()
)
```

Dealing with outliers in the year of registration column we use the earliest year of 1903 to compare if any outliers lie beyond that, if so we apply this mask and use a group by method to use features like (standard model, standard colour, standard make, body type) to compute a new year of registration to these outliers:

```
temp_years_imputation = (
    AT_copy.groupby(['standard_model','standard_colour','standard_make','body_type'])['year_of_registration']
    .transform('median')
)
mask = AT_copy['year_of_registration'] < 1903
AT_copy.loc[mask, 'year_of_registration'] = temp_years_imputation[mask]
```

a simple check with $AT\_copy['year\_of\_registration'].min()$ tells us the earliest year is 1909. Mileage uses same group by method for missing values computation, but only includes one imputation step:

```
AT_copy['mileage'] = AT_copy['mileage'].fillna(
    AT_copy.groupby(['standard_model', 'standard_make'])['mileage']
    .transform('median')
)
```

Similar methodologies have been implemented in imputing missing values in features standard colour, body type, and fuel type. An example of these methodologies is given below:

```
AT_copy['standard_colour'] = AT_copy['standard_colour'].fillna(
    AT_copy.groupby(['standard_model', 'standard_make'])['standard_colour']
    .transform(lambda x: x.mode()[0] if not x.mode().empty else np.nan)
)
global_mode = AT_copy['standard_colour'].mode()[0]
AT_copy['standard_colour'] = AT_copy['standard_colour'].fillna(
    AT_copy.groupby('standard_make')['standard_colour']
    .transform(lambda x: x.mode()[0] if not x.mode().empty else global_mode)
)
```

Difference being rather than 3 steps, 2 steps imputation is applied, with step 2 including global_mode fall back method.

2.2 Feature Engineering, Transformations, Feature Selection

Aforementioned in the Figure 1, data shows price is heavily skewed to the right with a long tail. Such skewed data negatively hinders the prediction capabilities of the model. To deal with this we can implement multitude of methods, one such method is removal of rows containing prices that are skewed. Doing so may potentially improve model scoring. Removal of these rows means loss of information on specific vehicle types, brand, models and market based information.

To counter this another method used is a Transformation method, specifically log Transformation (`log1p`) was appllied to the target variable. Such methods allow us to keep the data of price distribution shape in a symmetric form, while as mentioned in earlier section this method preserves the data integrity without any loss of information. Log transformation does not necessarily mean changing the meaning of the data itself but simply reducing the spread. The result is a more compressed tail section and price data being more manageable, the effect of which are visible in a histogram plot in Figure 2.

3 Model Building

3.1 Algorithm Selection and Configuration

Employment of the right type of method is crucial for prediction depending on the type of data this could make all the difference. Three models are implemented KNN Regressor, Decision Tree Regressor, and Linear regressor. Since the data contains numerical feature price as the Target, implementation of regressor type model makes sense.

1. **Decision Tree Regressor:** Its ability to capture no linearity makes it a viable approach. Considering the diversity of the dataset features numerical, binary, categorical features, Decision Tree Regressor for hierarchical design rule implementation.

2. **Linear Regressor:** A simplistic model that can capture linearity in the dataset. Can be used as a benchmark for potentially strong and more complex models.

3. **KNN Regressor:** A model that relies om similarity in feature space rather than relying on the relationship between features. This model serves to capture patterns in the dataset that could explain price feature behavior.

All models use **scikit-learn** Pipeline, this is to ensure consistency in pre-processing and preventing data leakage during the training phase. The phase includes, numerical scaling, categorical, and binary encoding. This step was implemented as:

```
num_pipe = Pipeline(steps=[
    ('scaler', StandardScaler())  # numeric features passed as-is
])
# Categorical pipeline: impute missing values, then one-hot encode
low_card_cat_pipe = Pipeline(steps=[
    ('onehot', OneHotEncoder(sparse_output=False,
    handle_unknown='infrequent_if_exist',  # groups rare categories
    drop='if_binary')),
])
```

```
high_card_cat_pipe = Pipeline(steps=[
     ('tar_enc', TargetEncoder())
])
binary_pipe = Pipeline(steps=[
    ('ordinal_enc', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1))
])
```

Configuration of Decision Tree Regressor included keeping *max_depth=12* while minimum sample leaf was configured to *min_samples_leaf=10*. Linear regressor model configuration remained simply as *LR = LinearRegression()*, while KNN regressor configuration included *n_neighbors=11*.

3.2 Grid Search, Model Ranking and Selection

The implementation of grid search required configuration for the Decision Tree Regressor model. Setting max depth from 1 to 12 and sample leaf from 10 to 20 this was done using the **sklearn.model_selection** library:

```
param_grid = {
    'tree_reg__max_depth': [ 1, 3, 4, 8, 10, 12],
    'tree_reg__min_samples_leaf': [ 10, 20, 30 ]
}
```

By using the **GridSearchCV** library, we implement Grid Search for hyperparameter tuning with cross-validation, applying a configured pipe into it after fitting the model.

```
gs = GridSearchCV(clf_pipe, param_grid, return_train_score=True)
```

After this a two step method is implemented for fitting and getting the result in a data frame form.

```
gs_results = gs.fit(X_train, y_train_log)
gs_df = pd.DataFrame(gs_results.cv_results_)
```

The top three results are then printed on the Table 2 and ranked by their score level.

max_depth	min_samples_leaf	mean_train	std_train	mean_test	std_test	rank
12	10	0.933	0.000350	0.922	0.001382	1
12	20	0.930	0.000394	0.921	0.001420	2
12	30	0.928	0.000206	0.920	0.001398	3

Table 2: Decision Tree Regressor Grid Search Results

This shows the top configuration for Decision Tree is *max_depth=12* and *min_samples_leaf=10*. Maximum depth here controls the expressive capability of the tree, with deeper depth of the trees captures more complex patterns at the risk of overfitting. Minimum samples per leaf regularizes the tree by preventing it from learning overly specific patterns from very small sample groups.

Model performance was assessed using cross-validated R^2 scores. Models were ranked based on their mean validation performance, and training scores were examined in parallel to diagnose potential overfitting. these performance measurements were done using:

```
scores_dict = cross_validate(clf_pipe, X_train, y_train_log, cv=5, return_train_score=True)
scores = pd.DataFrame(scores_dict)
print(scores['train_score'].mean(), scores['train_score'].std())
print(scores['test_score'].mean(), scores['test_score'].std())
```

with outputs scores:

Table 3: 5-Fold Cross-Validated R^2 Test Scores

Model	**Mean CV R^2**	**Std. Dev.**
Decision Tree Regressor	0.9229505534375392	0.0015447792011793594
Linear Regression	0.850235439079215	0.0025636020101361557
KNN Regressor	0.9298174092445086	0.0006226295222697311>

This Table 3 provides clear evidence of the best performing model with Linear Regressor in the third place and lowest nonperforming model among the three, and equal level of train and test scores. While in second place Decision Tree Regressor with minimum difference in the score, train test performance slightly higher than test score. In first place KNN Regressor shows the best performance. although drawbacks include the heavy computational requirements for the model and time consuming model fitting and evaluation which takes well over 15 minutes. In this case the better choice is Decision Tree Regressor as the best model.

4 Model Evaluation and Analysis

4.1 Coarse-Grained Evaluation

Model performance was first evaluated using global regression metrics. Table 4 gives a report on the training and test R^2 scores for all three models.

Model	**Train R^2**	**Test R^2**
Decision Tree Regressor	0.9335019255538202	0.925964152816406
Linear Regression	0.8521581348663565	0.853695673959911
KNN Regressor ($k = 11$)	0.9444753912682555	0.9336881336313425

Table 4: Train and Test R^2 Scores for Evaluated Models

The Decision Tree Regression's training performance indicates strong capacity to model non-linear relationships, with both training and test scores being mostly equal to each other with small differences. Linear Regression exhibits lowest performance but is more consistent across training and test sets with no difference, while KNN shows highest training and test scores among all three models.

4.2 Feature Importance

Feature importance was analyzed using the trained Decision Tree Regressor, which provides a measure of every feature's contribution in reducing prediction error in the model. The

method for implementation and analysis included using **scikit-learn** library feature **feature_importances_**.

```
importances = clf_pipe.named_steps['tree_reg'].feature_importances_
feature_names = clf_pipe.named_steps['preprocess'].get_feature_names_out()
feat_ = pd.DataFrame({
    'feature': feature_names,
    'importance': importances
}).sort_values('importance', ascending=False)

sns.barplot(
    data=feat_.head(10),
    x='importance',
    y='feature'
)
plt.title('Top 10 Feature Importances')
plt.show()
```

This method extracts the features after pre-processing and creates a sorted data-frame of these features for further analysis using a bar-plot. The Figure 10 confirms our initial belief of year of registration being the most important feature followed by standard, model mileage, standard make and lastly body type. Other model features do not visibly contribute in the model prediction.

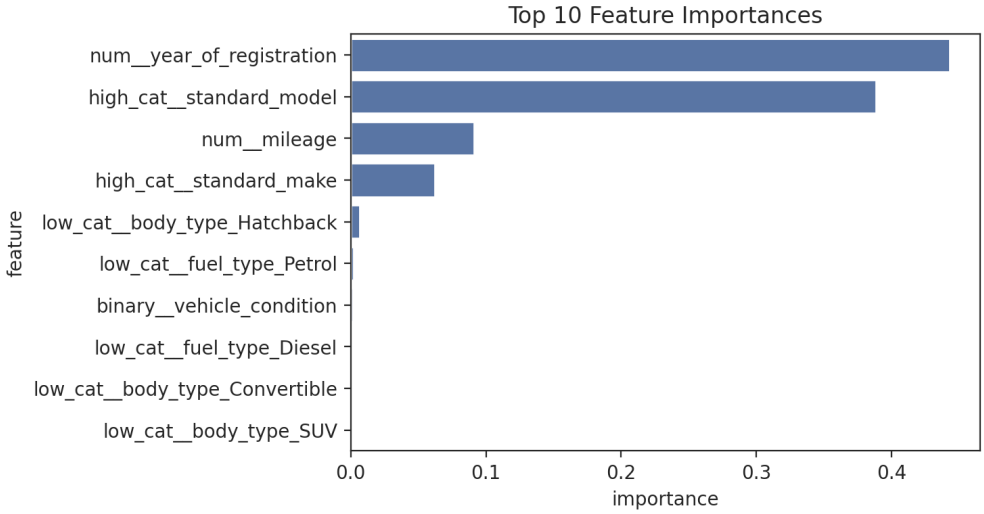


Figure 10: Important Features

4.3 Fine-Grained Evaluation

Fine-grained evaluation was conducted using prediction error analysis on the test set. Figure 11 and Figure 13 gives the actual versus predicted log-transformed prices (original scale prices) for the Linear Regression(LR) and KNN model, while Figure 14 and Figure 12 shows residuals plotted against predicted values. The implementation was was done using the library **sklearn.metrics** feature **PredictionErrorDisplay** shown below.

```
    PredictionErrorDisplay.from_estimator(
    clf_pipe,
    X_test, y_test_log, kind="actual_vs_predicted",
    scatter_kwargs=dict(alpha=0.5)
);
```

Residual distributions are centered around zero, indicating no bias in the system. However, variance in residuals increases which is most likely due to higher-priced vehicles, suggesting reduced accuracy in predicting extreme price values. This pattern is consistent with the skewed nature of price feature.
To quantify real-world prediction error, the original price scale was reversed through the log transformation method before Mean Absolute Error (MAE) was computed. The Decision Tree Regressor achieved an MAE score of 2721 compared to a baseline MAE of 10795 obtained by predicting the mean price. This demonstrates substantial improvement compared to the baseline predictor.

```
    y_pred = np.expm1(log_test)
    y_test_actual = np.expm1(y_test_log)
    mean_absolute_error(y_test_actual, y_pred)
    y_pred_baseline = y_test_actual.mean()
    y_pred_baseline
    np.mean(np.abs((y_test_actual - y_pred_baseline)))
```

Overall, fine-grained analysis shows that the selected models capture general pricing trends effectively, while at the same time exhibiting higher uncertainty for high-priced vehicles.

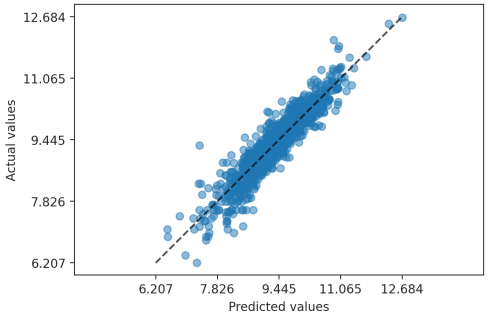


Figure 11: LR plot

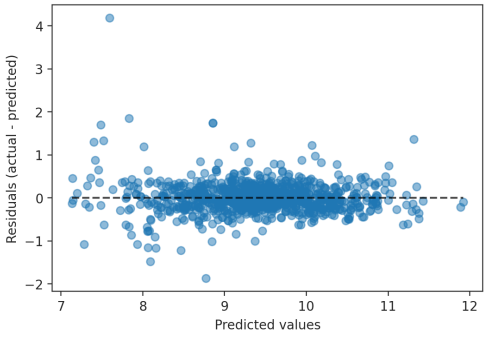


Figure 12: LR residuals

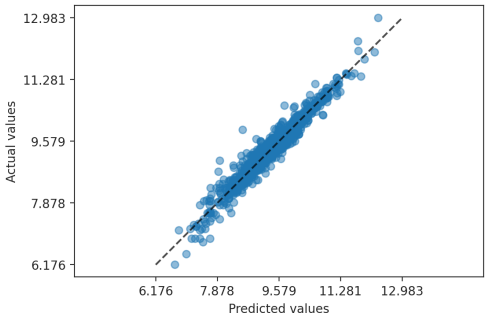


Figure 13: KNN versus plot

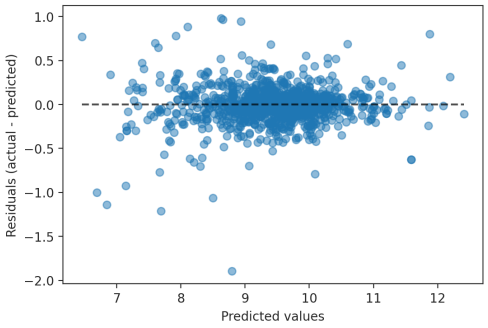


Figure 14: KNN residuals